

推定器



Thought for 47s >

結論から言うと：

1. 今回のエラーそのものは「仕方がない（理論上そうなる）」です。バイナリ処置前提の推定器に、実質 **1** クラス **or** **3** クラス以上の処置を入れているので、統計的に推定不可能。
2. 「仕方がない」で終わらせずに、**Causal Design** 画面で事前チェック + 人間語のガイドを出すべきです（今の `This solver needs samples of at least 2 classes...` は完全にライブラリ都合のメッセージ）。
3. 将来的には、**UI** 上で「処置のバイナリ化ウィザード」や **multi-arm** 対応を追加できると、ここが大きな差別化ポイントになります。

1. なぜこのエラーは避けられないのか

(1) 現行の推定器の前提

今の DR / IPW / DiD などの実装は、基本的に

- **Treatment** T : 0 = コントロール, 1 = 施策あり
- **Outcome** Y : 連続 or 0/1
- 「両方の群にサンプルがある」ことが前提 (Positivity)

で動いています。

そのため、

- `treatment` 列が **全部 0** か **全部 1**
- または、**文字列 3 クラス以上** (`standard/enhanced_care/digital_followup`) をそのまま入れる

と、

「少なくとも 2 クラス必要だが 1 クラスしかない」

という、ライブラリ (たぶん sklearn 系) レベルの例外が出ます。

これは**数学的にどうしようもない条件**なので、「コードをうまく書けば消える」という種類の話ではありません。

(2) 今回の `hospital_patient_outcomes` のケース

あなたがメモに書いている通りで、

- 元の列: `treatment_arm = standard/enhanced_care/digital_followup` (3 クラス)
- Causal Design v1 が想定: **0/1** のバイナリ処置

なので、今の実装では

1. 「ユニーク値数を数える」
2. `n_unique < 2` or `n_unique > 2` なら **明示的に CQOx 側のドメインエラー**を出す
3. そうしないと、下層のライブラリが勝手に `This solver needs samples of at least 2 classes...` を投げる

という構造になります。

あなたのスクショのようにまだ

「3 クラス以上 → バイナリに変換されていない」

ケースでライブラリの例外が出ているのは、**CQOx 側のバリデーションが 1 箇所足りていない**だけです。

(ここは単に追加すればよい)

2. 「仕方がない」前提で UI にどう書くか

ここは**プロダクトとしての UX**の問題なので、

「ユーザに“自分が悪いのか？ ツールが悪いのか？”を 1 秒で理解させる」ことがゴールです。

(A) Causal Design 画面: Treatment Column 選択時のリアルタイム警告

場所：

`Treatment Column` のドロップダウンのすぐ下 (今 `auto-detected` と出ている領域)

表示ロジック (バックエンド **or** フロントで事前チェック)

- `uniq = df[treatment_col].dropna().unique()`
- `n = len(uniq)`

```
def validate_treatment_column(df: pd.DataFrame, col: str) -> dict:
    values = df[col].dropna().unique()
    n = len(values)

    if n < 2:
        return {
            "status": "error_single_class",
            "values": [str(v) for v in values],
        }
    if n > 2:
        return {
            "status": "error_multiclass",
            "values": [str(v) for v in values],
        }
    return {"status": "ok", "values": [str(v) for v in values]}
```

(B) 表示するメッセージ案 (ユーザ向け)

ケース 1: 1 クラスしかない (今回のエラー)

処置列が 1 クラスしかありません

このデータでは、選択された処置列「insurance_type」に
実質 1 種類の値しか含まれていません (例: すべて「Basic」のみ)。

CQOx の因果推定 (DR / IPW など) は、
「施策なし (0)」と「施策あり (1)」という 2 つのグループを比較する前提です。
そのため、処置列には 少なくとも 2 クラス (0/1 など) が必要です。

対処方法

1. フィルタ条件を見直し、「施策あり／なし」の両方が入るようにする
2. もしくは、施策がまったく打たれていないデータの場合は、
この CSV では因果効果を推定できません (構造上の制約です)

ケース 2: 3 クラス以上ある (multi-arm)

現在は 0/1 処置のみ対応しています

選択された処置列「treatment_arm」には

["standard", "enhanced_care", "digital_followup"] のように 3 クラス以上の値が含まれています。

CQOx v1 の推定器は、「施策あり (1)」vs「施策なし (0)」のバイナリ処置のみ対応しています。

対処方法の例

- 例 1: `is_enhanced_care = 1 if treatment_arm == "enhanced_care" else 0` のような列を CSV に追加し、
その列を Treatment Column として選択してください。
- 例 2: すでに `icu_admission` や `readmission_30d` のような 0/1 列がある場合は、
それらを処置として選ぶこともできます (「ICU 入室あり／なし」を施策とみなす、など)。

画面下部のグローバル注意書き

Causal Design セクションの下に、薄めのインフォボックスで常に表示：

現在の制約 (v1)

- 処置は 0/1 のバイナリ列のみ対応しています。
- 3 クラス以上の処置や、まったく施策のないデータでは因果推定を行えません。
- 将来的には multi-arm (複数施策) のサポートを予定しています。

3. 実装側でやっておくといこと

(1) バックエンドでのバリデーションを「入口」でやる

今は推定器の中 (最深部) でライブラリが落ちているので、

FastAPI のエンドポイントに入った直後で弾くと UX がきれいになります。

擬似コード：

```
def run_causal_analysis(request: CausalRequest) -> CausalResult:
    df = load_dataset(request.dataset_id)
    validate_basic_shapes(df, request)
```

```

t_col = request.treatment_column
t_validation = validate_treatment_column(df, t_col)

if t_validation["status"] == "error_single_class":
    raise UserFacingError(
        code="treatment_single_class",
        message="処置列が 1 クラスしかありません。施策あり・なし両方が含まれる必要があります。",
        details=t_validation,
    )

if t_validation["status"] == "error_multiclass":
    raise UserFacingError(
        code="treatment_multiclass",
        message="現在は 0/1 のバイナリ処置のみ対応しています。3 クラス以上の処置はバイナリ列に変換してください。",
        details=t_validation,
    )

# ここから先で初めて estimator を呼び出す

```

フロントは `code` を見て、上で書いたメッセージを表示するだけにする、という構造。

(2) 「バイナリ化ウィザード」は v2 の強い拡張候補

v1.0 ではドキュメント+バリデーションだけで十分ですが、

v2 ではここをプロダクト価値に変えられます：

- multi-arm 処置列を選んだときにポップアップを出して、
 - 「どのクラスを“施策あり”とみなしますか？」
 - 「どのクラスをベースラインにしますか？」
 - 「enhanced_care vs その他 で 0/1 列を自動生成します」
- 自動生成した列は **CQOx 内部だけの仮想列**として扱う（元 CSV は触らない）
- Audit Log / Export Gate に「どのようにバイナリ化したか」を必ず書き出す

こうすると、

- 統計的な制約は守りつつ、
- 非エンジニアユーザでも **multi-arm 施策データ**を扱える

ので、既存の Haus / Incrmntal 群より一段 UX が上がります。

4. 「これは仕方ない？」への答えを整理すると

- 数学的には「仕方ない」制約です（2 クラス以上いないと因果効果は推定できない）。
- ただし、今のままライブラリの英語エラーをそのまま見せるのはプロダクト側の負けです。
- CQOx 側で
 - 「事前バリデーション」
 - 「人間語でのエラー表示」
 - 「どう直せばいいかの具体例（0/1 列の作り方）」
 を出せば、ユーザは混乱せずに済みます。

What this does（上のコード案について）

- **ゴール**
 - Causal Design の実行前に、「処置列が 0/1 バイナリで、かつ 2 クラスとも存在するか」を検査し、条件を満たさない場合は **CQOx 独自の分かりやすいエラーコード**で止める。
- **入力**
 - `df`: アップロード済みデータセット
 - `treatment_column`: ユーザが UI で選んだ処置列名
- **出力・副作用**
 - 問題なしなら何も返さず推定処理へ進む。
 - 問題があれば `UserFacingError(code=..., message=..., details=...)` を投げ、フロントでは `code` ごとに日本語のエラーメッセージと対処方法を表示。
- **なぜこのアプローチか**

- ライブラリ由来の曖昧な英語エラーを隠し、
「データ構造上の制約」であることをユーザに明示するため。
- 将来 multi-arm 対応やウィザードを追加するときも、
この `code` ベースの分岐にフックを追加すればよく、拡張しやすい。
- 動作確認の方法 (例)

```
# 1. 単一クラスの treatment 列を持つ CSV で API を叩く
curl -X POST http://localhost:8000/api/causal/run \
-F dataset_id=hospital_patient_outcomes_1M \
-F treatment_column=insurance_type \
-F outcome_column=... \
-F estimators='["dr", "ipw"]'

# レスポンス JSON に code="treatment_single_class" が含まれることを確認

# 2. 3 クラス以上の treatment 列で同様にテスト
# code="treatment_multiclass" が返ることを確認
```

Expert insight (Google/Meta/NASA level)

「統計モデルが受け取れるデータの条件を、UI テキストではなく *型システム* として表現する」のが大規模プロダクトでは重要です。
データ検証を estimator の奥ではなく、**API 境界・スキーマ・専用の validator レイヤ**に寄せておくと、

- エラーは常に同じコード体系で返せる
- ログ・監査・アラートも集約できる
- 将来のモデル差し替え時も UX を変えずに済む
というメリットが出ます。

もしよければ、今書いている日本語の説明文を UI 向けにもう一段だけ短く・カード化した形にリライトもできます。